

QuickDelivery - Documentação

1. Proposta do Domínio

1.1 Descrição

O **QuickDelivery** é uma plataforma de entregas sob demanda que conecta dois perfis distintos de usuário: o **cliente**, que solicita o transporte de um item entre dois endereços, e o **prestador de serviços** (entregador), que aceita a demanda e executa a entrega. O sistema implementa o ciclo de vida da solicitação: criação, aceite, execução, conclusão ou cancelamento.

1.2 Justificativa

- **Distinção clara entre cliente e prestador.** Cliente e entregador possuem papéis, permissões e fluxos diferentes.
- **Domínio simples e verificável**, permitindo foco nos requisitos arquiteturais: REST, persistência, autenticação, autorização e organização em camadas.
- **Escopo controlado**, com fluxo principal curto: criar -> aceitar -> em andamento -> entregue.

1.3 Perfis de Usuário

Cliente. Solicita entregas, lista suas próprias solicitações, acompanha status e pode cancelar uma entrega antes da execução.

Prestador de Serviços (entregador). Visualiza entregas pendentes, aceita demandas, acompanha entregas atribuídas a ele e atualiza o status da execução.

1.4 Principais Funcionalidades

Cliente

- Criar conta e autenticar.
- Solicitar entrega com origem, destino e descrição.
- Listar e consultar apenas suas próprias entregas.
- Cancelar entrega permitida pela máquina de estados.

Entregador

- Criar conta e autenticar.
- Listar entregas pendentes.
- Aceitar entrega usando seu próprio `deliverymanId`.
- Atualizar status para `IN_PROGRESS` e `DELIVERED`.

2. Backend REST

O backend expõe endpoints REST para autenticação, consulta de usuários por perfil e operação das entregas. O modelo de usuário foi unificado em `users`, com papel `CUSTOMER` ou `DELIVERYMAN`, substituindo a separação inicial entre clientes e prestadores. As entregas referenciam `customerId` e, após aceite, `deliverymanId`.

A API permite cadastrar usuários, autenticar com email e senha, consultar o usuário autenticado, listar clientes e entregadores, criar entregas para o cliente autenticado, listar entregas conforme o perfil, consultar uma entrega por ID e atualizar o status da entrega com validação de transição. As respostas usam códigos HTTP convencionais: `201` para criação, `200` para leitura ou atualização, `400` para validação, `401` para autenticação ausente ou inválida, `403` para ação proibida e `404` para entidades inexistentes. Erros seguem o formato `{ "error": "mensagem" }`.

3. Segurança e Autorização

O backend usa autenticação por token no padrão JWT assinado com HMAC SHA-256. O token contém `userId`, `role`, data de emissão e expiração. O middleware de autenticação interpreta o header `Authorization: Bearer <token>` e injeta o usuário autenticado na requisição.

Regras principais:

- Cliente só cria entregas para si mesmo.
- Cliente só lista, consulta e altera suas próprias entregas.
- Entregador vê entregas pendentes e entregas atribuídas a ele.
- Entregador só pode aceitar entrega usando seu próprio `deliverymanId`.
- Dados públicos de cliente/entregador não expõem senha.

4. Persistência

O schema Prisma usa PostgreSQL e contém as tabelas `users` e `deliveries`. A tabela `users` armazena dados de autenticação, contato e papel do usuário. A tabela `deliveries` representa a solicitação de entrega, com cliente obrigatório, entregador opcional, endereços, descrição e status.

5. Mensageria

O backend publica eventos no RabbitMQ após operações importantes do ciclo de entregas. Ao criar uma entrega, publica `delivery.created`; ao aceitar uma entrega, publica `delivery.accepted`; e a cada mudança de status publica `delivery.status_changed`. Um consumidor de exemplo lê a fila `quickdelivery.delivery-events` para demonstrar o processamento assíncrono sem chamada REST direta.

6. Organização do Código

O backend mantém separação em camadas. As rotas mapeiam verbos e URLs para os controllers, os controllers lidam com `req` e `res` e delegam a regra de negócio para os services, os services concentram validações, autorização e máquina de estados, os repositories encapsulam o acesso ao Prisma, os middlewares cuidam de

autenticação e tratamento centralizado de erros, e os types concentram tipos compartilhados de usuário e status de entrega.

7. Validação via Postman

A coleção `postman/QuickDelivery.postman_collection.json` cobre o fluxo autenticado. Antes de executá-la, é necessário rodar:

```
npm run seed
```

A coleção realiza login do cliente e do entregador, armazena os tokens em variáveis, cria uma entrega e percorre o fluxo `PENDING -> ACCEPTED -> IN_PROGRESS -> DELIVERED`.